120 minutos · guía del expositor

Explica una idea. Demuéstrala enseguida.

La teoría presenta el mapa. La práctica muestra cómo cada pieza se convierte en algo que funciona en Android.

El ritmo de la sesión

Minutos	Qué haces	Señal para avanzar
0–5	Objetivo, materiales y clave OMDb	Todos abrieron la práctica.
5–17	Teoría esencial y descargas en paralelo	Pueden explicar WebView, puente y plugin.
17–35	Preparar Mac y app base	App Cordova abierta en Android.
35–45	1. Pantalla y SQLite	Mensaje “Tu catálogo está listo”.
45–60	2. Búsqueda	Inception y detalle visibles.
60–75	3. Guardado	Una pendiente y una vista persisten.
75–87	4. Catálogo	Tarjetas separadas por estado.
87–99	5. Edición	Cambio guardado y eliminación confirmada.
99–107	6. Recomendación	Película recomendada y guardada.
107–120	Prueba final, recuperación y dudas	Catálogo conservado al reabrir.

Antes de mañana: prueba una Mac de la escuela, permisos y red; solicita la clave OMDb; confirma Java 17, SDK 36 y Gradle. La instalación de herramientas no cabe siempre en 18 minutos. Ten tu emulador listo y conserva el material descargado como respaldo.

0–5 · Abre con el resultado

> “Hoy vamos a construir una app Android con HTML, CSS y JavaScript. No vamos a convertir cada botón en código Java: Cordova mete nuestra interfaz web dentro de una aplicación y nos permite usar funciones nativas con plugins.”

Muestra: tu QueVer terminado brevemente, sin entregar sus archivos. Busca una película, guárdala y vuelve al catálogo. Después abre la práctica desde el inicio.

Pide: abrir el enlace del taller y solicitar o activar la clave OMDb. Si ya pueden iniciar descargas necesarias, déjalas corriendo durante teoría.

Pregunta de arranque: "¿Qué diferencia hay entre visitar una página y tener una app instalada?"

5–17 · El mapa de Cordova

Abre las diapositivas. Las notas del presentador se abren con `S`; las flechas cambian de slide y `F` activa pantalla completa.

> "HTML dice qué aparece; CSS cómo se ve; JavaScript qué pasa. La WebView ejecuta esa parte web. El puente comunica JavaScript con código nativo. Un plugin implementa una capacidad concreta: en nuestro caso, SQLite."

Historia en un minuto: PhoneGap nace en Nitobi en 2008; Adobe adquiere Nitobi en 2011; la base llega a Apache y adopta el nombre Cordova. PhoneGap cerró en 2020, pero no es lo mismo que Cordova.

Ejemplo concreto: buscar películas con `fetch` es una llamada web; guardar con `window.sqlitePlugin` cruza el puente hacia SQLite en Android.

No afirmes: que Cordova convierte HTML en controles nativos, que una sola base elimina toda configuración por plataforma o que todo plugin funciona en cualquier sistema.

Comprueba: "¿Para consultar OMDb necesito el plugin SQLite?" Respuesta: no; solo lo usamos para la base local.

17–35 · Que Android abra algo antes de programar

> "Cordova es el coordinador. Node y npm nos permiten instalarlo; Java, el SDK y Gradle construyen la app Android. Android Studio ya nos ofrece el emulador, pero la terminal también debe encontrar esas herramientas."

Proyecta: Preparar la Mac. Explica la diferencia entre consultar una versión e instalar algo. Usa el AVD que ya existe.

Descompón el comando: `cordova create`, carpeta, identificador y nombre visible. `platform add` añade Android; `plugin add` añade SQLite; `requirements` diagnostica; `run` compila y despliega.

Puerta de avance: app inicial abierta en el emulador. Si un equipo no llega por permisos o red, que continúe temporalmente en pareja; no abras una segunda ruta con iOS o navegador.

35–45 · Fase 1: esperar al dispositivo

> "Primero vamos a tener una pantalla y comprobar que podemos hablar con SQLite. No queremos pedirle algo al teléfono antes de que Cordova esté listo: para eso existe deviceready."

Abre: fase 1. Explica una responsabilidad por archivo. Copia los archivos completos, configura tu clave sin proyectarla y guarda.

Señala: `document.addEventListener`, `initDB()` y `CREATE TABLE IF NOT EXISTS`. El esquema incluye los campos futuros para evitar migraciones; todavía no incluimos funciones de guardado.

Prueba: ejecutar y esperar "Tu catálogo está listo". No uses una captura del catálogo final como resultado de esta fase.

45–60 · Fase 2: el viaje de una búsqueda

> "Al enviar el formulario, leemos el título, hacemos una petición HTTPS y recibimos JSON. Después creamos elementos HTML para mostrarlo. Pedir el detalle es otra consulta usando un identificador IMDb."

Abre: fase 2. Usa "Ir al siguiente cambio" para explicar el código nuevo, pero copia siempre el archivo completo.

Señala: `submit → searchMovies → requestOmdb → renderSearchResults`. Recalca que `config.local.js` se conserva.

Prueba: Inception, abrir detalle y búsqueda vacía. Si falla la clave, revisar activación y cuota; no reinstalar Android.

60–75 · Fase 3: de remoto a local

> "OMDb conoce la película. Nuestra base conoce si yo la vi y qué opiné. Vamos a unir ambas cosas y guardar una copia en este emulador."

Abre: fase 3. Señala el paso `saveMovie → insertMovie → SQLite → listado`. Explica `?` como parámetro y `imdb_id` como identidad única.

Prueba: una pendiente y una vista, intentar duplicado, cerrar y reabrir. No continúes si guardar solo modifica la pantalla pero no persiste.

Pregunta: "Si guardo aquí, ¿aparece automáticamente en otro teléfono?" No: cada instalación tiene su base local.

75–87 · Fase 4: datos y presentación son distintos

> "Ya tenemos los datos. Ahora vamos a ordenarlos sin cambiar de base: una sección de pendientes, otra de vistas y tarjetas que podemos abrir."

Abre: fase 4. Señala la consulta por estado y el renderizado. Explica que "Ver más" amplía una lista ya consultada; no es paginación de la API.

Prueba: conserva las películas anteriores y abre una tarjeta. Si no da tiempo a guardar seis, explica el límite de cinco sin simular una prueba que no hiciste.

87–99 · Fase 5: cerrar el CRUD

> "Crear y leer ya funcionan. Ahora añadimos actualizar y eliminar. La pantalla solo debe mostrar éxito cuando la base confirma el cambio."

Abre: fase 5. Relaciona `INSERT / SELECT / UPDATE / DELETE` con los botones. Explica por qué eliminar pide confirmación.

Prueba: editar opinión, pendiente a vista, cancelar eliminación y luego confirmarla. Reinicia para verificar que el cambio vive en la base.

99–107 · Fase 6: reutilizar, no duplicar

> "La recomendación no es inteligencia artificial. Elegimos una búsqueda y un resultado al azar. Lo interesante es que, aunque la entrada es distinta, reutilizamos el mismo guardado."

Abre: fase 6. Señala la selección aleatoria, petición de detalle y llamada a `saveMovie()`.

Prueba: recomendar y guardar; cerrar otra sin guardar. Si la red falla, mostrar el aviso, no inventar una recomendación.

107–120 · Cierra con evidencia, no con una captura

> "Acabamos de usar tres capas: la web para la interfaz y las consultas, Cordova para empaquetar y conectar, y Android con SQLite para los datos locales. Ahora cerramos y abrimos la app para demostrar que no era solo un cambio visual."

Haz: la comprobación final. Pide que alguien explique el recorrido de guardar una película.

Recupera: si alguien se atrasó, copia los archivos completos de la fase actual y los de recuperación, conservando su clave. Un fallo de instalación requiere diagnóstico, no más código.

Cierre honesto: Cordova permite reutilizar conocimientos web; seguimos dependiendo de las herramientas Android, de la red para OMDb y del mantenimiento de los plugins.

Respuestas rápidas para el salón

- **¿Es una página web?** La interfaz usa tecnología web, pero está empaquetada como aplicación Android.
- **¿Necesita internet siempre?** Para buscar y recomendar sí; para consultar lo guardado, no.
- **¿Por qué Android Studio?** Nos aporta el SDK y el emulador; editamos el código compartido en VS Code.
- **¿Por qué copiar todo?** Para evitar errores de inserción. El resaltado nos deja explicar solo las diferencias.
- **¿La clave en JS es secreta?** Separarla evita publicarla por accidente en el repositorio, pero una app distribuida no puede esconder una clave embebida. Para un servicio con secretos reales se necesita un backend.

José Adrián Bazaldúa Huerta y Haili Ailin Ávila Ramírez · 9 de noviembre de 2026 · Universidad Iberoamericana.